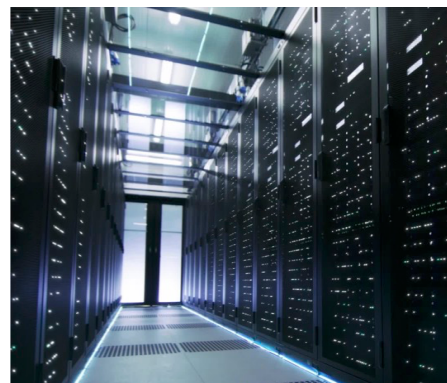


CS250: Foundations of Computer Systems

Computer Science Department, Colorado State University

CS250 covers foundational aspects of computer systems encompassing processors, networks, storage, and computing frameworks. A holistic understanding these constructs is key to designing systems (AI, Big Data, clouds, and security) that scale by surfacing problems and solutions from different vantage points. The course includes a discussion of processors, cores, and co-processors (GPUs). Speed differentials across the memory hierarchy and the implications of caching will be covered as well. The discussion of the memory subsystem is complemented with the design of data structures for storage systems. Core foundations of networking including the design of the key elements of the networking stack (IP, TCP, and UDP) will be covered alongside a discussion of DNS. The course also explores characteristics of I/O systems with an emphasis on on-disk data structures that facilitate effective data retrievals. Finally, the course ends with a future gazing component that explores new, promising systems on the horizon with an emphasis on neuromorphic computing



Course Objectives

Upon successful completion of this course students will be able to:

1. Explain core computer systems terminology
2. Explain binary number representations, signed numbers, and floating-point numbers alongside conversions across numbering systems
3. Synthesize concepts in Boolean logic and algebra
4. Distill and build upon core concepts in memory management
5. Describe the interplay between CPU, memory, the I/O subsystem alongside the execution of programs.
6. Design programs that leverage networking protocols that underpin the internet including IP, TCP, and UDP.
7. Design and implement core data structures used for storage systems
8. Explain the inner workings of Graphic Processing Units (GPUs)

Modules within CS250

Introduction

This module introduces students to why computer systems thinking is critical in the design of large-scale systems. Computer systems allow viewing the problem space from multiple vantage points. Such vantage points allow designers to come up with creative solutions to problems.

Objectives:

- [1] Summarize basic computer systems concepts
- [2] Identify key issues in computer systems

Binary number representations

The realm of computer systems is based on binary i.e., all operations are based on 1s and 0s. We will look at the rationale behind this decision and the efficiencies that this choice enables. We will look at properties of binary numbers and how to perform conversions between decimal and binary number representations.

Objectives:

- [1] Discuss rationale for binary number representations
- [2] Explain properties of binary numbers
- [3] Perform calculations that convert from binary to decimal and vice versa

Signed numbers and floating-point representations

Working with positive and negative numbers and exponents comes very naturally to us when we are dealing with decimal numbers. Turns out doing this in binary is not that straightforward. There are no “signs” or “powers” that are built into binary. We need to work with 1s and 0s to accomplish all our objectives. In this module we will be looking at approaches to represent numbers that are positive, negative, gargantuan, or miniscule.

Objectives:

1. Discuss the need for signed number representations
2. Explain and compute Two’s and One’s complement number representations of negative numbers.
3. Identify challenges in representation of gargantuan/miniscule numbers using binary
4. Explain the single-precision and double-precision floating point representations that can be used to represent both gargantuan and miniscule numbers.

Boolean Logic and Algebra

Boolean logic is the bedrock of computer systems: the language used to make decisions. If regular algebra is the mathematics of numbers, Boolean algebra is the mathematics of true and false. We’ll start by exploring how to derive truth tables from Boolean functions and, in reverse, how to construct Boolean functions from truth tables. Along the way, we’ll break down the essential building blocks of digital logic -- elementary gates and their corresponding truth tables. And finally, we’ll see something remarkable: how every Boolean function, no matter how complex, can be built using nothing but NAND gates.

Objectives:

1. Derive truth tables from Boolean functions
2. Explain and apply De Morgan’s Laws to transform Boolean expressions
3. Explain elementary gates: Or, And, Not, Xor, Nand, and Nor
4. Synthesize Boolean functions from Truth Tables
5. Prove how all Boolean functions can be constructed using only NAND gates

Memory Basics

When we write programs, we rely on variables, arrays, and data structures to store, modify, and retrieve values. But underneath it all, everything comes down to memory -- how state is stored, accessed, and changed over time. We’ll explore the mechanics of storing, retrieving, and modifying state, peeling back

the layers to see what's really happening in memory. Then we'll tackle a deeper question: how do circuits keep track of time? To do that, we'll introduce clocks, the beating heart of digital systems, which help us model the progression of time in computation.

Objectives:

1. Explain how progression of time is modeled and its implication on storage of state
2. Identify and explain differences in Combinational and Sequential logic
3. Describe the data flip flop

Memory Hierarchy

The speed differential between different levels of memory can make or break a computer's performance. A well-designed system carefully manages these differences, ensuring that data moves efficiently between registers, L1/L2/L3 caches, and main memory. We'll explore how these layers work together to keep access times low, and we'll take a deep dive into cache design: examining direct-mapped caches, fully associative caches, and N-way associative caches to see how they help bridge the gap between blazing-fast processors and slower memory.

Objectives:

1. Explain organization of the memory hierarchy encompassing registers, L1/L2/L3 caches, and main memory.
2. Describe memory addressing schemes and how word-aligned memory accesses occur
3. Synthesize concepts of cache lines and differences in direct-mapped and associative caching schemes
4. Design programs that identify and profile the impact of different caching based either on temporal or spatial locality

Networking Concepts

Think of communication networks as enabling conversations at lightning speed. In this module, we'll unravel some of the fundamental ideas that make these conversations possible. First, we'll examine how data is prepared for its journey: how raw information is encoded before being sent across a network. Then, we'll dive into the two main types of networks: circuit-switched and packet-switched. We will also explore how multiple conversations can share the same infrastructure, a concept known as *multiplexing*. Finally, we'll explore one of the most elegant principles in networking: encapsulation. Much like the way letters are placed in envelopes, addressed, and sent through layers of postal systems, encapsulation ensures that messages travel efficiently and reach the right destination. Understanding these core ideas will give us a deeper appreciation for the choreography that underpins modern communication.

Objectives:

1. Describe data encoding and representation mechanisms in communications
2. Explain differences in circuit switched networks and packet switched networks
3. Identify how multiplexing of data can occur over shared networks
4. Understand organization principle of protocol layers and encapsulation

Internet Architecture

This module is at the heart of our discussions on networking, where we unravel the elegant design of the Internet Protocol (IP) stack. At its core is a layered architecture that keeps the internet running smoothly. We'll examine the foundation of it all -- the Internet Protocol itself -- exploring both IPv4, the workhorse of today's internet (and yesteryears), and IPv6, its forward-looking upgrade designed for the future. We'll also contrast two very different approaches to communication: TCP, which ensures messages arrive reliably and in order, and UDP, which trades reliability for speed. Together, these ideas reveal the underpinnings of every connection, from web browsing to video streaming.

Objectives:

1. Identify the rationale for layering and the role it plays in encapsulation
2. Explain fragmentation and reassembly in IP networks
3. Explain the inner workings of IPv4 and IPv6 alongside key architectural differences that exist between them
4. Identify key differences in connection-oriented and connectionless communications
5. Explain key concepts underpinning TCP such as congestion control, flow control, and the sliding window protocol including setup and teardown of TCP connections.
6. Design and develop programs that leverage networked communications.

Networking Extras

In this module we look at additional critical concepts in networking. In particular, we look at issues such as how ISPs cope with the limited availability of unique IPv4 addresses, how several of these design decisions also help address security issues, and how hosts are able to discover hosts over the internet. Our discussions in this module encompass the interrelated issues of private IP addresses, network address translation, and the domain name services.

Objectives:

1. Describe how network address translations (NATs) work
2. Highlight the need and rationale for private addresses
3. Distill core ideas in DNS and how network addresses are resolved

The von Neumann Computing architecture

Computers designed over the past 70 years or so are all based on the von Neumann architecture. In this module, we will look at several elements that comprise this architecture. Also included in this module is the notion of the stored program concept -- one of the most foundational ideas in computer science. The module will also cover aspects relating to bus architectures, Moore's law, and Dennard's scaling. A key concept that we also look at in this module is Memory-Mapped I/O which allows a computer system to interoperate with a plethora of Input/Output (I/O) devices.

Objectives:

1. Explain the stored program concept
2. Identify different constructs within the von Neumann architecture and how they have influenced design of computer systems

	3. Explain how memory mapped I/O allows the computer system to reconcile diverse I/O devices 4. Explain differences between the Harvard architecture and the von Neumann architecture 5. Explain the role buses play in the von Neumann architecture.

Machine Language

Programs expressed in high-level languages need to be converted to a representation that the computer system can execute. This includes translating the semantics of higher-level languages into those that the computer system can process. We will be looking at characteristics of machine languages both binary and symbolic. We will also be looking at the notion of Instruction Set Architectures (ISAs) and we will be looking at different generations of the x86 ISA and ARM.

	Objectives: 1. Explain difference between binary and symbolic versions of machine language. 2. Identify the role of ISA as a model for how computations are performed. 3. Explain evolution of the x86 ISAs over time.

Software

In this module we will be looking at how programs expressed in high-level languages are processed. In particular, we will be looking at the notion of a virtual machine (used in languages such as Java and C#) and stack machines. We will also be looking at the notion of heaps, stacks, and stack frames that play a crucial role in program execution and garbage collection.

	Objectives: 1. Explain the two-tiered compilation model in virtual machine environments 2. Explain the role (and interactions between) stacks, heaps, and stack frames 3. Describe the stack machine

Data Structures for Storage Systems

In this module, we look at a key data structure for one of the most important I/O device: stable storage. We describe the key differences between in-memory and on-disk data structures. To ensure that our discussions are self-contained we first start with linear scan that we then improve by pruning search spaces using binary search. We discuss the notion of dynamic data structures that adapt to contents of the data items. Next, we cover a popular in-memory data structure, Binary Search Trees (BSTs). We then cover one of the most foundational data structures for storage systems: B-Trees. In particular, B-trees are data structures that are designed to align with how data access and transfers (block-based) take place in storage systems. We highlight both the similarities and key differences between BSTs and B-Trees. Issues such as insertions, deletions, merges, and balancing in B-Trees are discussed as well. We complete our discussions on B-Trees with variants such as B*-trees and B+-Trees.

	Objectives: 1. Identify how binary search improves upon linear scan 2. Explain key concepts underlying dynamic data structures 3. Design and develop programs that implement BSTs including insertions, deletions, and search 4. Design and develop programs that implement B-Trees including insertions, deletions, and search

Graphics processing units (GPUs)

Graphics processing units (GPUs) are used extensively in gaming and AI/ML (Artificial Intelligence/Machine Learning) applications. In this module, we will explore GPUs and contrast them with CPU based systems. A key emphasis here is to identify the specific types of problems that GPUs are particularly well-suited for. We will explore the issue of memory locality and cache coherency in both GPU and CPU based systems. Notably, CPUs and GPUs work in tandem with each other and workloads that effectively target them will see considerably improved performance.

	Objectives: 1. Explain the role of the GPU in the computing ecosystem 2. Identify how the GPU differs from (and complements) the CPU

Readings will be based on the following recommended texts.

- [NS] *The Elements of Computing Systems, second edition: Building a Modern Computer from First Principles*. 2nd Edition. Noam Nisan and Shimon Schocken. ISBN-10/ ISBN-13: 0262539802 / 978-0262539807. MIT Press.
- [MJ] *How Computers Really Work: A Hands-On Guide to the Inner Workings of the Machine*. Matthew Justice. ISBN-10/ISBN-13 : 1718500661 / 978-1718500662. No Starch Press.
- [JS] *The Secret Life of Programs: Understand Computers -- Craft Better Code*. Jonathan E. Steinhart. ISBN-10/ ISBN-13 : 1593279701 / 978-1593279707. No Starch Press.
- [DT] Peter J. Denning and Matti Tedre. Computational Thinking. Essential Knowledge series. The MIT Press. ISBN-10:ISBN-13 0262536560/ 978-0262536561. 2019. [Chapter 2]

Grading

The weights associated with different elements of the course are listed below.

Course Element	Weight
----------------	--------

Assignments (4)	30% [7.5, 7.5, 7.5, 7.5]
Labs/recitations	10%
Quizzes	10%
Midterm I	15%
Midterm II	15%
Final Exam	20%

Letter grades will be based on the following standard breakpoints: ≥ 90 is an A, ≥ 88 is an A-, ≥ 86 is a B+, ≥ 80 is a B, ≥ 78 is a B-, ≥ 76 is a C+, ≥ 70 is a C, ≥ 60 is a D, and < 60 is an F.

About the Professor

Shrideep Pallickara is a Professor of Computer Science at Colorado State University where he also directs the Center for eXascale Spatial Data Analytics and Computing (XSD). His research explores the design of large-scale computing systems for big data, distributed analytics, and GeoAI helping to shape the ways in which we can illuminate the dynamics of complex systems. He has published over 175 peer-reviewed articles. His research has been supported by major funding agencies in both the United States and the United Kingdom, including the National Science Foundation's flagship programs in scientific innovation, computing systems, and artificial intelligence; the Department of Homeland Security; the National Institute of Food and Agriculture; the U.K. EPSRC's e-Science program; and the National Endowment for the Humanities. His research and assorted systems software have been deployed in domains such as epidemiology, precision agriculture, urban resilience, earthquake science, environmental and ecological monitoring, and high-energy physics. He is a recipient of the *Board of Governor's* award, the NSF CAREER award, and a Monfort Professorship.